sql/query_json_operators.sql

```
-- water was offered
EXPLAIN SELECT * FROM trips
WHERE data @> '{"ride_details":{"water_offered": true}}';

-- 2 bags in the trunk
EXPLAIN SELECT * FROM trips
WHERE data @> '{"ride_details":{"bags_in_trunk": 2}}';
```

You'll continue to expand your usage of these new analytics about trips.

Correlate these ride analytics with particular drivers and locations to help answer Rideshare operational questions. To do that you'll explore B-tree indexes with JSON data. B-tree Indexes can be used with JSON data by creating an expression as you did earlier.

Run the following statement in psql to create a B-Tree Index on the data column for expressions on specific attributes. In the case below, the index narrows down all trips where two or more bags in the trunk were recorded.

```
CREATE INDEX ON trips USING btree(data)
WHERE (data->'ride_details'->'bags_in_trunk')::int4 >= 2;
```

With that index in place, the following query can be run efficiently. Confirm the index you just added was used by prepending EXPLAIN. Run \d trips to confirm the index names correlate to the Indexes you expect.

```
EXPLAIN SELECT * FROM trips
WHERE (data->'ride_details'->'bags_in_trunk')::int4 >= 2;
```

When deciding on JSONB with Ruby on Rails for row storage, consider using the store_accessor functionality for the Active Record model described in *Hidden Gems: ActiveRecord Store storage*.[8]

One trade-off to be aware of with Store Accessor is that it's limited to a single level of depth. The above structure where bags_in_trunk is nested under ride_details wouldn't work because of more than one level of depth. For that, use the JSONB capabilities directly from Active Record.

You've now seen how you can use GIN indexes for use cases like efficiently looking up values in JSONB columns.

What else can GIN indexes be used for?

Using GIN Indexes for Full Text Search

You can use a GIN index to provide full text search capabilities to Rideshare.

8. https://www.honeybadger.io/blog/rails-activerecord-store/